

Templates

This document is written in Markdown.

If it looks unstyled here, view it on GitHub for a formatted version.

Templates let you define what fields appear and in what order. They are plain JSON files you can edit in any text editor.

Templates control layout only. They cannot change app behavior or data.

1) The mental model (two modes)

Dataset mode

Dataset mode is for big flat arrays (table-like). Templates do not apply in this mode.

Records mode

Records mode is for structured objects and API responses. Templates control layout and record labels here.

If you are not seeing template effects, first check your mode.

2) The basic workflow

1. In JTF, click **Download** in the Templates panel to get a starter template.
2. Edit it (paths and labels). Templates can be saved as `.json` or `.jsonc` (JSON with comments).
3. Upload the template back into JTF.
4. Select it in the Template dropdown.

Tip: If your browser shows the JSON instead of downloading it, use Save As and store it locally.

3) Template anatomy (what's inside)

A template contains:

- `templateName`
- Optional `match` rules (controls when it applies)
- Optional `recordLabel` rules (controls the Record dropdown label)
- `layout`: sections and fields (controls what you see)

If a path does not exist, it is skipped. Nothing crashes.

4) Paths: how JTF finds your values

Paths are dot-separated:

- `Status.Name`

- `AssignedTo.Email`
- `ViewerUrl`

JTF reads the record and walks the path. If anything is missing along the way, the field does not render.

Practical tip:

- Start by copying real property names from the Raw JSON view.
- Build one field at a time.

5) Layout: sections and fields

`layout` is an ordered list of sections.

Each section has an ordered list of fields.

That is the whole trick.

Example shape:

- Section: "Header"
 - Field: path + label + format
- Section: "Details"
 - Field: path + label + format

6) Formats (how values display)

Common formats used in templates:

- `text`: default
- `badge`: compact label pill (great for status values)
- `date`: readable date/time
- `link`: clickable URL
- `multiline`: preserves line breaks
- `json`: shows a nested object or array (can be collapsible)
- `chips`: renders an array of values as compact badges (great for people, teams, disciplines, recipients)
- `kvlist`: renders an array of objects as a compact key → value list, where each item becomes a single labeled row

`kvlist` resolves values in priority order.

If multiple value properties exist (for example dropdown values and text values), JTF chooses the first non-empty value.

If a value is the wrong type for a format, the field is skipped.

Badge styles

Badges support style hints that help visually group similar values.

Style	Typical use
<code>meta</code>	identifiers, IDs
<code>info</code>	descriptive attributes
<code>location</code>	place or grouping
<code>money</code>	numeric values
<code>status</code>	workflow or priority

Example:

```
{
  "path": "Status.Name",
  "label": "Status",
  "format": "badge",
  "style": "status"
}
```

Styles affect appearance only.

If no style is provided, the default badge styling is used.

Colors adapt automatically to light and dark themes.

Chips

`chips` expects an array most of the time, but JTF can also handle “object-or-array” shapes (single object treated like a 1-item array).

Use it for things like:

- Notify users / teams
- Disciplines
- Recipient lists
- Simple attachment lists (file names)

JSON (cards for arrays)

`json` normally renders nested data with collapsible viewing.

If the field includes a `fields` list and the value is an array, JTF renders it as a set of compact cards (one card per array item). This is how Comments, Attachments, and History can be readable without dumping raw JSON.

Example:



```
{
  "path": "Attachments",
  "label": "Attachments",
  "format": "json",
  "fields": [
    { "path": "Name", "label": "Name", "format": "text" },
    { "path": "Date", "label": "Date", "format": "date" },
    { "path": "Size", "label": "Size", "format": "text" }
  ]
}
```

Files templates (special case)

Files exports include version history and activity events.

To keep these readable, Files templates commonly use:

- `json` with `fields` to render history as cards
- `kvlist` to flatten property/value arrays (for example version numbers)

This avoids deeply nested dropdowns while keeping all data visible.

7) Match rules (what they do, and what they do not)

Match rules decide whether a template applies to a given record.

Auto template selection

If the template dropdown is set to **Auto (best match)**, JTF picks the best matching template for each record while you browse in **Records** mode.

Important notes:

- Templates apply in **Records** mode only (Dataset mode never uses templates).
- JTF match rules check **top-level keys and values only**. Nested paths (for example `listItem.type.name`) are not supported for matching.
- If nothing matches, JTF falls back to its built-in views.

Match rules are intentionally simple.

Match is **not** filtering.

Match is **not** a query language.

Match does **not** transform data.

Keep match rules small and predictable.

Plain Export vs Combined Export

Some tools export the same type of data in different shapes.

Example: RFIs

Plain export

Top-level keys like:

- `number`
- `subject`
- `question`

Combined export

Wrapped records with top-level keys like:

- `rfiId`
- `listItem`
- `payloads`
- `project`

These shapes require **different templates**.

If a template technically matches but shows little or no data, the most common cause is using the **right template with the wrong export shape**.

For Newforma Konekt data (Issues, RFIs, Submittals, Change Items, Files, Emails, and Viewer Files), Plain and Combined exports always require **separate templates**. Viewer Files templates cover 3D models (IFC) and sheets viewed in the NFK Viewer created on or before Aug. 19, 2024. Files created after that date use the standard Files templates.

Good match rule examples

Combined Change Items

```
"match": {  
  "requiredKeys": ["payloads", "listItem", "changeItemId"]  
}
```

This works because `changeItemId` exists only on Change Item exports and is a top-level key.

Combined RFIs / Submittals / Issues

```
"match": {  
  "requiredKeys": ["payloads", "listItem"]  
}
```

Use additional top-level keys (such as `rfiId`, `issueId`, `submittalId`) when you need to distinguish between record types.

Common mistakes

- Using nested paths in match rules (for example `listItem.type.name`)
- Reusing a Plain template for a Combined export
- Over-specifying match rules when a single unique key is enough

When in doubt:

- Match on **one or two unique top-level keys**
- Let layout paths handle the rest

8) Record labels (the Record dropdown)

Templates can define how records are labeled in the **Record** dropdown. This is one of the most useful (and most overlooked) parts of a template.

A record label is built from one or more fields, evaluated **in order**.

If a field does not exist or is empty, JTF skips it and moves on.

If none of the fields produce a value, JTF uses the fallback label.

Basic example

```
"recordLabel": {
  "fields": [
    { "path": "Number", "prefix": "#", "maxLen": 10 },
    { "path": "Title", "prefix": " ", "maxLen": 60 }
  ],
  "fallback": "Record {n}"
}
```

This turns a generic label like:

Record 12

Into something meaningful:

#ISSUE-1042 Door clearance

Combining multiple fields

You can combine several fields to create richer labels:

```
"recordLabel": {
  "fields": [
    { "path": "Number", "prefix": "#", "maxLen": 10 },
    { "path": "Subject", "prefix": " ", "maxLen": 60 },
    {
      "path": "Status.Name",
      "prefix": " (",
      "suffix": ")",
      "maxLen": 20
    }
  ],
  "fallback": "Item {n}"
}
```

Example output:

```
#RFI-231 Window detail (Open)
```

Record labels in Combined exports

Combined exports usually wrap summary fields under a `listItem` object.

Record labels work the same way, but the paths must point to the combined shape:

```
"recordLabel": {
  "fields": [
    { "path": "listItem.number", "maxLen": 20 },
    { "path": "listItem.subject", "prefix": " • ", "maxLen": 80 },
    {
      "path": "listItem.status.name",
      "prefix": " (",
      "suffix": ")",
      "maxLen": 20
    }
  ],
  "fallback": "Item {n}"
}
```

Example output:

This pattern is commonly used for:

- Combined RFIs
- Combined Submittals
- Combined Change Items
- Combined Issues

If a combined record is not showing a label, double-check that the paths use `listItem.*` (or the correct combined wrapper) rather than plain export paths.

Prefixes, suffixes, and limits

Each label field supports:

- `prefix`: text added before the value
- `suffix`: text added after the value
- `maxLen`: maximum characters to keep
- `lastChars`: keep the last *N* characters (useful for IDs)

Example:

```
{ "path": "Id", "prefix": " [", "suffix": "]", "lastChars": 8 }
```

Which renders like:

```
[9f3a2c1b]
```

Fallback behavior

If none of the label fields resolve to a value, JTF uses the fallback:

```
"fallback": "Record {n}"
```

`{n}` is replaced with the record index.

This guarantees the dropdown is never empty or confusing.

Best practices

- Use short, stable fields (numbers, titles, subjects)
- Avoid long text or descriptions
- Include status when it helps scanning
- Do not rely on optional fields unless you provide a good fallback

Good record labels make browsing large datasets faster and far less frustrating.

9) When a template matches but shows nothing

Sometimes a template matches a record, but no fields render.

This usually means one of the following:

- You are in **Dataset mode** instead of **Records mode**
- A field path is wrong (typo, casing, or different property name)
- The template is designed for a **different record shape** (for example, combined export vs plain export)

When this happens, JTF shows a friendly hint and still allows access to the raw record view.

Nothing is broken. The template just does not line up with the data.

10) Guardrails (important)

Templates are **declarative only**.

- No scripting
- No conditional logic
- No data mutation

Templates describe **how data is displayed**, not how it behaves.

JTF reads data.

It does not change data.